Lab-Assignment-11.5

Name : Sampelly Suhas

Ht.no :2303A51453 Bt.no: 21

Course :Ai Assistant Coding

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty

methods.

Sample Input Code:

class Stack:

pass

Expected Output:

• A functional stack implementation with all required methods and

Docstrings

```
class Stack:
    """
    A Stack data structure implementation using a Python list.
    """
    pass
```

```
class Stack:
    """
    A Stack data structure that supports push, pop, peek, and is_empty operations.
    """

    def __init__(self):
        """
        Initialize an empty stack.
        """
        self.items = []
```

```python
    def push(self, item):
        """
        Push an item onto the top of the stack.

        Parameters:
        item : any
            The element to be added to the stack.
        """
        self.items.append(item)
```

```python
    def pop(self):
        """
        Remove and return the top item from the stack.

        Returns:
        any
            The top element of the stack.

        Raises:
        IndexError
            If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Pop from empty stack")
        return self.items.pop()
```

```python
def peek(self):
    """
    Return the top item without removing it.

    Returns:
    any
        The top element of the stack.

    Raises:
    IndexError
        If the stack is empty.
    """
    if self.is_empty():
        raise IndexError("Peek from empty stack")
    return self.items[-1]
```

```python
def is_empty(self):
    """
    Check whether the stack is empty.

    Returns:
    bool
        True if the stack is empty, False otherwise.
    """
    return len(self.items) == 0
```

```python
# Test Code
s = Stack()
s.push(10)
s.push(20)
print(s.pop())
print(s.peek())
print(s.is_empty())
```

```
20
10
False
PS C:\Users\sampe\Downloads>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size Methods

```python
class Queue:
    """
    A Queue data structure implementation using a Python list.
    """
    pass


class Queue:
    """
    A FIFO-based Queue that supports enqueue, dequeue, peek, and size operations.
    """

    def __init__(self):
        """
        Initialize an empty queue.
        """
        self.items = []
```

```python
def enqueue(self, item):
    """
    Add an item to the rear of the queue.

    Parameters:
    item : any
        The element to be added to the queue.
    """
    self.items.append(item)
```

```python
def dequeue(self):
    """
    Remove and return the front item of the queue.

    Returns:
    any
        The front element of the queue.

    Raises:
    IndexError
        If the queue is empty.
    """
    if self.size() == 0:
        raise IndexError("Dequeue from empty queue")
    return self.items.pop(0)
```

```python
    def peek(self):
        """
        Return the front item without removing it.

        Returns:
        any
            The front element of the queue.

        Raises:
        IndexError
            If the queue is empty.
        """
        if self.size() == 0:
            raise IndexError("Peek from empty queue")
        return self.items[0]
```

```python
    def size(self):
        """
        Return the number of elements in the queue.

        Returns:
        int
            The size of the queue.
        """
        return len(self.items)
```

```python
# Test Code
q = Queue()
q.enqueue(10)
q.enqueue(20)
print(q.dequeue())
print(q.peek())
print(q.size())
```

```
10
20
1
PS C:\Users\sampe\Downloads>
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method documentation.

```
class Node:
    """
    Represents a node in a singly linked list.
    """
    pass
```

```
class LinkedList:
    """
    A Singly Linked List implementation with insert and display methods.
    """

    def __init__(self):
        """
        Initialize an empty linked list.
        """
        self.head = None
```

```python
def insert(self, data):
    """
    Insert a new node at the end of the linked list.

    Parameters:
    data : any
        The data to be inserted into the linked list.
    """
    new_node = Node()
    new_node.data = data
    new_node.next = None

    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
```

```python
    def display(self):
        """
        Display all elements of the linked list.
        """
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")


# Test Code
ll = LinkedList()
ll.insert(10)
ll.insert(20)
ll.insert(30)
ll.display()
```

```
10 -> 20 -> 30 -> None
PS C:\Users\sampe\Downloads>
```

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

```python
class HashTable:
    """
    A Hash Table implementation using chaining for collision handling.
    """
    pass


class HashTable:
    """
    Hash table that supports insert, search, and delete operations
    using chaining to handle collisions.
    """

    def __init__(self, size=10):
        """
        Initialize the hash table.

        Parameters:
        size : int
            Number of buckets in the hash table.
        """
        self.size = size
        self.table = [[] for _ in range(size)]
```

```python
    def _hash(self, key):
        """
        Compute hash index for a given key.
        """
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table.
        Uses chaining to handle collisions.
        """
        index = self._hash(key)

        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return
```

```python
    def search(self, key):
        """
        Search for a value using its key.
        Returns the value if found, otherwise None.
        """
        index = self._hash(key)

        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]
        return None
```

```python
    def delete(self, key):
        """
        Delete a key-value pair from the hash table.
        Returns True if successful, otherwise False.
        """
        index = self._hash(key)

        for i, pair in enumerate(self.table[index]):
            if pair[0] == key:
                self.table[index].pop(i)
                return True
        return False
```

```python
# Test Code
ht = HashTable()

ht.insert("apple", 100)
ht.insert("banana", 200)
ht.insert("orange", 300)

print(ht.search("banana"))
print(ht.delete("banana"))
print(ht.search("banana"))
```

```
200
True
None
PS C:\Users\sampe\Downloads>
```

Task Description #5 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

• **Graph with methods to add vertices, add edges, and display connections.**

```python
class Graph:
    """
    Graph implementation using an adjacency list.
    """
    pass
```

```python
class Graph:
    """
    A graph represented using an adjacency list.
    Supports adding vertices, adding edges, and displaying connections.
    """

    def __init__(self):
        """
        Initialize an empty graph.
        """
        self.graph = {}

    def add_vertex(self, vertex):
        """
        Add a vertex to the graph.

        Parameters:
        vertex : any
            The vertex to be added.
        """
        if vertex not in self.graph:
            self.graph[vertex] = []
```

```python
def add_edge(self, vertex1, vertex2):
    """
    Add an edge between two vertices (undirected graph).

    Parameters:
    vertex1 : any
        First vertex
    vertex2 : any
        Second vertex
    """
    if vertex1 not in self.graph:
        self.add_vertex(vertex1)
    if vertex2 not in self.graph:
        self.add_vertex(vertex2)

    self.graph[vertex1].append(vertex2)
    self.graph[vertex2].append(vertex1)
```

```python
class Graph:

    def display(self):
        """
        Display the adjacency list of the graph.
        """
        for vertex in self.graph:
            print(vertex, "->", self.graph[vertex])


# Test Code
g = Graph()

g.add_vertex("A")
g.add_vertex("B")
g.add_vertex("C")

g.add_edge("A", "B")
g.add_edge("A", "C")
g.add_edge("B", "C")

g.display()
```

```
A -> ['B', 'C']
B -> ['A', 'C']
C -> ['A', 'B']
PS C:\Users\sampe\Downloads>
```

Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.

3. Medical Records Storage – Fast retrieval of patient details using ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

1. Patient Check-In System → Queue

Justification:
Patients must be treated in the same order in which they arrive. A Queue follows the FIFO (First In First Out) principle, ensuring fairness in handling regular patient registrations and treatment flow.

---

2. Emergency Case Handling → Priority Queue

Justification:
Emergency patients must be treated based on the severity of their condition rather than arrival time. A Priority Queue ensures that patients with critical conditions are treated first.

---

3. Medical Records Storage → Hash Table

Justification:
Medical records need to be retrieved quickly using a unique patient ID. A Hash Table allows fast insertion and retrieval in constant time on average, making it efficient for managing patient data.

---

4. Doctor Appointment Scheduling → Binary Search Tree (BST)

Justification:
Appointments must be stored in sorted order based on time. A Binary Search Tree automatically maintains sorted order, making it easy to

insert, search, and display appointments chronologically.

---

5. Hospital Room Navigation → Graph

Justification:
Hospital wards and rooms are connected through corridors like a network. A Graph represents rooms as nodes and pathways as edges, enabling efficient navigation and route planning.

```python
import heapq

class EmergencyQueue:
    """
    Priority Queue implementation for handling emergency patients.
    Patients with higher priority are treated first.
    """

    def __init__(self):
        """
        Initialize an empty emergency queue.
        """
        self.queue = []

  def add_patient(self, priority, patient_name):
      """
      Add a patient to the emergency queue.

      Parameters:
      priority : int
          Priority of the patient (lower number = higher priority)
      patient_name : str
          Name of the patient
      """
      heapq.heappush(self.queue, (priority, patient_name))
```

```python
    def treat_patient(self):
        """
        Remove and return the highest priority patient.

        Returns:
        tuple
            Patient priority and name
        """
        if not self.queue:
            return "No patients in emergency queue"
        return heapq.heappop(self.queue)
```

```python
    def display_queue(self):
        """
        Display all patients in the emergency queue.
        """
        print("Emergency Queue:", self.queue)


# Test Code
eq = EmergencyQueue()
eq.add_patient(1, "Patient A")
eq.add_patient(3, "Patient B")
eq.add_patient(2, "Patient C")

print(eq.treat_patient())
print(eq.treat_patient())
```

```
(1, 'Patient A')
(2, 'Patient C')
PS C:\Users\sampe\Downloads>
```

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

- **For each feature, select the most appropriate data structure from the list below:**
  - **Stack**
  - **Queue**
  - **Priority Queue**
  - **Linked List**
  - **Binary Search Tree (BST)**
  - **Graph**
  - **Hash Table**
  - **Deque**
- **Justify your choice in 2–3 sentences per feature.**
- **Implement one selected feature as a working Python program with AI-assisted code generation.**

Expected Output:

- **A table mapping feature → chosen data structure → justification.**
- **A functional Python program implementing the chosen feature with comments and docstrings.**

1. Traffic Signal Queue → Queue

Justification:
Vehicles waiting at traffic signals must be processed in the order in which they arrive. A Queue follows the FIFO (First In First Out) principle, ensuring smooth and fair movement of traffic.

---

2. Emergency Vehicle Priority Handling → Priority Queue

Justification:
Emergency vehicles such as ambulances and fire trucks must be given priority over regular vehicles. A Priority Queue ensures that vehicles with higher urgency are allowed to pass first, regardless of arrival order.

---

3. Vehicle Registration Lookup → Hash Table

Justification:
Vehicle details must be accessed instantly using the registration number. A Hash Table provides fast insertion and retrieval in constant time on average, making it ideal for real-time lookup.

---

4. Road Network Mapping → Graph

Justification:
Roads and intersections form a connected network structure. A Graph represents intersections as nodes and roads as edges, enabling efficient navigation and route planning.

---

5. Parking Slot Availability → Deque

Justification:
Parking slots may be allocated and released from both entry and exit points. A Deque allows efficient insertion and deletion from both ends, making it suitable for tracking parking slot availability.

```python
import heapq

class EmergencyTrafficQueue:
    """
    Priority Queue implementation for handling emergency vehicles.
    Vehicles with higher priority are allowed to pass first.
    """
```

```python
    def __init__(self):
        """
        Initialize an empty emergency traffic queue.
        """
        self.queue = []

    def add_vehicle(self, priority, vehicle_name):
        """
        Add a vehicle to the priority queue.

        Parameters:
        priority : int
            Priority of the vehicle (lower number = higher priority)
        vehicle_name : str
            Name or ID of the vehicle
        """
        heapq.heappush(self.queue, (priority, vehicle_name))
```

```python
    def allow_vehicle(self):
        """
        Remove and return the highest priority vehicle.

        Returns:
        tuple or str
            Vehicle details or message if queue is empty.
        """
        if not self.queue:
            return "No vehicles waiting"
        return heapq.heappop(self.queue)

    def display_queue(self):
        """
        Display all vehicles in the priority queue.
        """
        print("Emergency Vehicle Queue:", self.queue)
```

```python
# Test Code
etq = EmergencyTrafficQueue()
etq.add_vehicle(1, "Ambulance")
etq.add_vehicle(3, "Police Car")
etq.add_vehicle(2, "Fire Truck")

print(etq.allow_vehicle())
print(etq.allow_vehicle())
```

```
PS C:\Users\sampe\Downloads>  & 'c:\Users\sampe\AppData\Local\Programs\Python\Python313\python.exe' '
c:\Users\sampe\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher
' '64454' '--' 'C:\Users\sampe\Downloads\AiAC 11.5(1453).py'
(1, 'Ambulance')
(2, 'Fire Truck')
```

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System

with:

1. Shopping Cart Management – Add and remove products dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

1. Shopping Cart Management → Linked List

Justification:
Products in a shopping cart are frequently added and removed dynamically. A Linked List allows efficient insertion and deletion of items without shifting elements, making it suitable for managing cart operations.

---

2. Order Processing System → Queue

Justification:
Orders must be processed in the same order in which they are placed. A Queue follows the FIFO (First In First Out) principle, ensuring fair and systematic order processing.

---

3. Top-Selling Products Tracker → Priority Queue

Justification:
Products need to be ranked based on their sales count. A Priority Queue keeps the highest-selling products at the top, allowing quick access to trending items.

---

4. Product Search Engine → Hash Table

Justification:
Product details must be retrieved quickly using a unique product ID. A Hash Table enables fast insertion and lookup in constant time on average, making it ideal for search operations.

---

5. Delivery Route Planning → Graph

Justification:
Warehouses and delivery locations are connected like a transportation network. A Graph represents locations as nodes and routes as edges, enabling efficient route planning and navigation.

```python
class CartNode:
    """
    Represents a product node in the shopping cart.
    """

    def __init__(self, product_name):
        """
        Initialize a cart node.

        Parameters:
        product_name : str
            Name of the product
        """
        self.product = product_name
        self.next = None
```

```python
class ShoppingCart:
    """
    Shopping Cart implemented using a Singly Linked List.
    Supports adding and removing products dynamically.
    """

    def __init__(self):
        """
        Initialize an empty shopping cart.
        """
        self.head = None
```

```python
def add_product(self, product_name):
    """
    Add a product to the shopping cart.

    Parameters:
    product_name : str
        Name of the product to add
    """
    new_node = CartNode(product_name)
    new_node.next = self.head
    self.head = new_node
```

```python
def remove_product(self, product_name):
    """
    Remove a product from the shopping cart.

    Parameters:
    product_name : str
        Name of the product to remove
    """
    temp = self.head
    prev = None

    while temp:
        if temp.product == product_name:
            if prev:
                prev.next = temp.next
            else:
                self.head = temp.next
            return True
        prev = temp
        temp = temp.next
    return False
```

```python
    def display_cart(self):
        """
        Display all products in the shopping cart.
        """
        temp = self.head
        print("Shopping Cart:")
        while temp:
            print("-", temp.product)
            temp = temp.next


# Test Code
cart = ShoppingCart()
cart.add_product("Laptop")
cart.add_product("Mobile Phone")
cart.add_product("Headphones")

cart.display_cart()
cart.remove_product("Mobile Phone")
cart.display_cart()
```

```
Shopping Cart:
- Headphones
- Mobile Phone
- Laptop
Shopping Cart:
- Headphones
- Laptop
PS C:\Users\sampe\Downloads>
```